Journal of Computational Science 57 (2022) 101508

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

Remaining useful life prediction in prognostics using multi-scale sequence and Long Short-Term Memory network★☆

Ruiguan Lin, Yaowei Yu, Huawei Wang *, Changchang Che, Xiaomei Ni

College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, China

ARTICLE INFO

Keywords:
RUL prediction
PHM
LSTM
Multi-scale sequence
Time window

ABSTRACT

The development of sensors and artificial intelligence technology provides practical tools for aircraft Prognosis and Health Management (PHM). The remaining useful life (RUL) prediction is the critical process of PHM. A novel data-driven framework is proposed to estimate the RUL of complex systems in this paper. The framework evaluates the system's RUL based on multi-scale sequences and Long Short-Term Memory (LSTM) networks. First, the sliding time window method is used to prepare training samples, and the degradation features are directly mapped to RUL predictions. In addition, the model parameters are adjusted through the input multi-scale sequence to obtain the best prediction performance. This method integrates the application of time window, multi-scale sequence, and LSTM structure to improve prediction accuracy. The proposed method is validated using the NASA C-MAPSS data set, and the results demonstrate the superiority of the proposed framework.

1. Introduction

With the latest developments in sensor technology and significant advances in machine learning technologies, traditional Condition-based Maintenance (CBM) maintenance strategies have become increasingly ineffective and outdated [1]. CBM cannot solve many problems such as aircraft use and operation support through predicting in advance, nor can it effectively support the collaborative optimization objectives of civil aircraft operation safety and cost control. Industry 4.0 technology based on Cyber-Physical Systems (CPS) provides a powerful tool for solving such problems and implementing Predictive Maintenance (PDM) [2–4]. The core of PDM is to carry out the RUL prediction [5]. RUL prediction is a technical term used to predict the future failure time or the remaining time to maintain regular operation [6]. Increasing the accuracy of RUL predictions can improve safety and reliability and extend operating time, thereby reducing average maintenance costs.

In the past few years, data-driven methods based on artificial intelligence have received more and more attention in the field of RUL prediction. Many machine learning techniques (especially deep learning) have been developed and have been successfully applied to predict the RUL of various complex systems. Classic deep learning algorithms include Deep Belief Network (DBN), Convolutional Neural Network (CNN), Stacked Auto Encoder (SAE), and Recurrent Neural Network (RNN) [7]. At present, the RUL prediction process of devices based on deep learning is generally as follows: first, data preprocessing is performed, including the division of training and test sets and data standardization. Second, use standardized sensor data to describe the degradation trend: (1) construct a Health Factor (HI) curve; (2) train the model to obtain the HI curve; (3) input the test set into the trained model to calculate relevant evaluation indicators and compare with existing methods.

In the field of deep learning, the network structure of RNN contains a recursive hidden layer, which is very suitable for time sequence processing, so it suits RUL prediction well. Hence it has caught much attention. Guo et al. [8] proposed a HI based on a recurrent neural network used for bearing RUL prediction. Liu et al. [9] proposed a fault diagnosis method for rolling bearings with automatic encoders based on recurrent neural networks. Because RNN can remember the past input information in a period, it is applicable for this task. However, RNN neural networks, which are prone to vanishing gradients and gradient

☆ The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author.
E-mail address: wang_hw66@163.com (H. Wang).

https://doi.org/10.1016/j.jocs.2021.101508
Received 17 January 2021; Received in revised form 2 November 2021; Accepted 22 November 2021
Available online 25 November 2021
1877-7503/© 2021 Elsevier B.V. All rights reserved.

explosions when dealing with long-term sequences [10,11], have poor accuracy. To overcome the weakness of RNN, this paper will use Long Short-Term Memory, which introduces input gate, output gate, forget gate, and cell state into RNN [12].

In 1997, Hochreiter and Schmidhuber [13] first proposed the LSTM model to solve the gradient disappearance problem of the traditional RNN model. Then, Graves [14] recently improved and popularized LSTM. LSTM can save long-term memory to the united-state and has achieved great success in many tasks. In recent years, LSTM has been widely used in the field of RUL prediction. Hinchi and Tkiouat [15] proposed a deep RUL estimation framework based on convolution and long short-term memory. First, the convolutional layer is applied to extract the local features of the sensor data directly. Then the LSTM layer is employed to obtain the degradation process and estimate the RUL. Yuan et al. [16] used the LSTM network to predict the RUL of the engine in the case of complex operation, mixed faults, and intense noise. Laredo et al. [17] proposed a multi-layer Perceptron (MLP) framework and evolutionary algorithms. They used a step-by-step time window and a piecewise linear model to estimate the RUL of mechanical components. Al-Dulaimi [18] proposed a hybrid deep neural network model for RUL estimation. This method takes LSTM paths to extract temporal features and CNN to extract spatial features simultaneously, which has an excellent prognostic effect on complex systems. Cao et al. [19] proposed a temporal convolutional network (TCN-RSA) with a residual self-attention mechanism to achieve end-to-end RUL prediction. According to the predicted health indicators, the main idea is to employ the constructed health factor and target markers to train the neural network model by setting the threshold to obtain the RUL estimation.

The research above of RUL prediction can be summarized in two ways: (1) Extrapolation of HI curve trend; (2) Direct mapping to RUL value. HI curve trend extrapolation, when the HI value exceeds a preset threshold it is deemed a failure, and then RUL can be calculated, is a multi-step backward prediction of the original HI curve. In contrast, the scheme that maps the HI curve to the RUL value directly establishes a direct mapping from input data to the RUL value without setting a failure threshold, which significantly reduces manual involvement to improve the efficiency and accuracy of the model prediction. Therefore, this paper adopts the method of direct mapping to RUL value for RUL prediction of turbine engines. In addition, the impact of sequence length on model performance is not considered in the existing methods, so this is one of the focuses of this paper.

This paper proposes a novel framework to estimate the RUL of complex systems. The framework represented as MS-LSTM evaluates the RUL of the system based on multi-scale sequences and LSTM. In the data preprocessing stage, a sliding time window method is used to prepare the data labels of the training set and the test set. The length of the input sequence is defined to give full play to the advantages of the LSTM model in processing long-term sequences; In the model construction, a structure of two hidden layers is adopted, and a dropout is set behind each hidden layer to prevent overfitting; In the model training, the EarlyStopping mechanism in Callbacks is added, preventing the model from overfitting by stopping the training in advance and obtaining the best test performance. In this paper, the RUL of an aero-engine is taken as an example. The publicly available NASA C-MAPSS dataset is used to verify the method's effectiveness and comparison with the latest research on the same dataset shows the superiority of the proposed framework. The main contributions of this paper are summarized as follows:

1) A deep LSTM network with double hidden layers is constructed to learn time dependence from time series sensor data, enabling the prediction network to remember useful degradation information effectively. At the same time, add Dropout behind each hidden layer to prevent overfitting.
2) A method for obtaining multi-scale time series feature information by adjusting the sequence length is proposed, which helps the network search for global information and get useful time information.
3) Based on the above, this paper proposes a framework based on MS-LSTM to achieve accurate RUL prediction of aero engines. This framework gives full play to the advantages of the LSTM network in processing long-term sequences and is conducive to enhancing the RUL prediction ability.

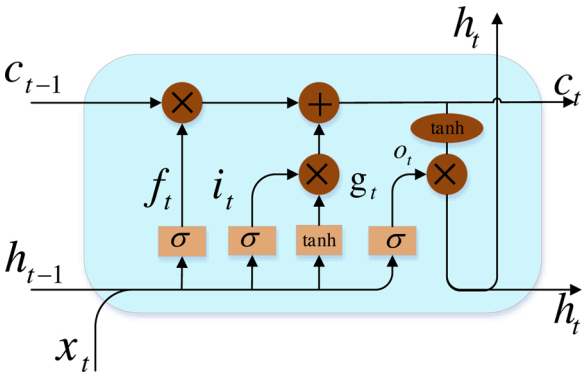


Fig. 1. The cell structure of LSTM.

The remainder of the paper is structured as follows: In Section 2, the MS-LSTM framework for RUL prediction is proposed. In Section 3, the performance of the proposed method is verified with the NASA C-MAPSS dataset. Finally, the conclusions and further works are drawn in Section 4.

2. Design of RUL prediction algorithm based on multi-scale sequence and LSTM

This section will introduce the proposed algorithm architecture for RUL prediction and its essential parts, including LSTM structure, RUL prediction model framework, and evaluation indicators based on multi-scale sequence and LSTM.

2.1. LSTM structure

2.1.1. LSTM network

In standard RNN, due to the frequent problems of vanishing gradient and explosive gradient, it is challenging to optimize model parameters through gradient dissipation [8]. In this paper, the LSTM network has been proposed to solve this problem. As a representative deep learning structure, LSTM is very powerful in discovering the changing patterns behind time series and is often used to deal with the long-term dependence of time series data [19].

Fig. 1 illustrates the structure of the LSTM layer at time t, which provides an affluent internal state by utilizing the cell state c_t, hidden state h_t and various gates. Suppose that N engines of the same type are monitored by m sensors during operation. Then, each component i i $=$ 1, 2, …,N, in the life cycle of sensor can be represented as a matrix: $\mathbf{X}_i = [\mathbf{x}^1, \mathbf{x}^2, \cdots, \mathbf{x}^N]$, $\mathrm{X}_i \in \mathbb{R}^{m\times N}$, where $\mathbf{x}^t = [x_1^t, x_2^t, \cdots, x_m^t]$ is the sensor measurement value at time t, and N represents the sequence length. Each gate that gets the current input layer data $\mathbf{X}_i$ and the previously hidden layer data h_{t-1} contains a Sigmoid function keeping the output value within [0,1], which determines whether to turn it on.

First, the forget gate f_t controls which long-term memories of the LSTM layer can be forgotten [20]:

$$f_i = \sigma(W_f h_{t-1} + U_f \mathbf{x}^t + b_f) \tag{1}$$

Next, the input gate calculates the information that can be obtained from the input and understands which parts should be stored in the cell state [21]:

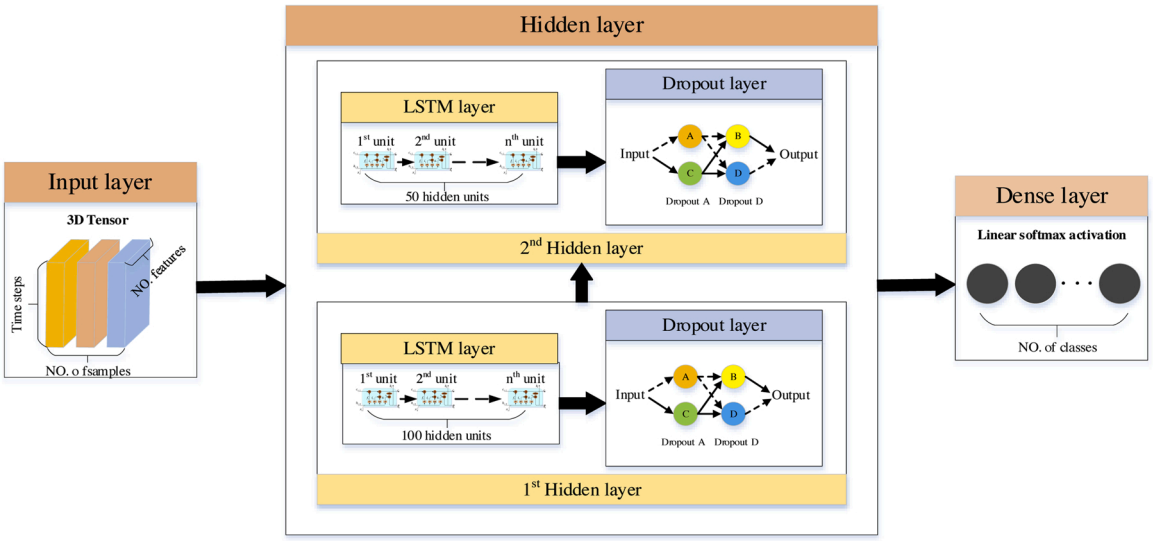


Fig. 2. Proposed deep LSTM model.

$$g_t = \tanh(W_g h_{t-1} + U_g \mathbf{x}^t + b_g) \quad (2)$$

$$i_t = \sigma(W_i h_{t-1} + U_i \mathbf{x}^t + b_i) \quad (3)$$

Then, update the long-term memory in the unit state:

$$c_t = c_{t-1} \otimes f_t + g_t \otimes i_t \quad (4)$$

Finally, the output gate updates the state of the current hidden layer based on the input, cell state, and previous hidden state [22]:

$$o_t = \sigma(W_o h_{t-1} + U_o \mathbf{x}^t + b_o) \quad (5)$$

$$h_t = o_t \otimes \tanh(c_t) \quad (6)$$

In the formula above, W_i, W_f, W_g, W_o are the hidden layer weight values between the current hidden layer and the previously hidden layer, and U_i, U_f, U_g, U_o are the weight values between the current input layer and the current hidden layer; b_i, b_f, b_g, b_o are the deviation vector; $\otimes$ is the element-wise multiplication operator; σ is the Sigmoid function; tanh is the activation function.

Behind the output layer is a neuron in the proposed model. The neuron can establish a direct mapping from degenerate features to RUL prediction based on given LSTM hidden state and prediction time t_i:

$$\widehat{RUL} = \sigma_r(W_r[h_t, t_i] + b_r) \quad (7)$$

Where σ_r is a linear activation function, (W_r, b_r) is the weight matrix and deviation.

2.1.2. Proposed deep LSTM architecture

The deep LSTM network regression model proposed in this paper is constructed by the Python keras/tensorflow deep learning library. Fig. 2 shows three layers in the deep LSTM: input, hidden, and output layers.

First, the input layer is the network layer that brings data into LSTM. The input data is in 3D format, namely [sample, time step, feature]. Time step represents the number of cells inside the LSTM unit, the value of which is commonly called sequence_length(SL) in keras/tensorflow. The impact of sequence_length on model training and prediction performance is the focus of this paper. For example, features include run settings, degradation cycles, and sensors for a specific numbered engine. During 192 degradation cycles, 21 sensors are used for condition monitoring, and the number of run setting variables is 3. Then, record the sensor data once at the end of each cycle, and the value of sequence_length of LSTM is 50, then the number of samples at this time = 192−50 + 1 = 143. Then the 3D format of the corresponding input data is [143, 50, 25].

Then, the hidden layer, the core part of model training and testing, lies between the input and output layers. In the hidden layer, the first layer is LSTM layer with 100 cells, and then another the LSTM layer is with 50 cells. In particular, Dropout is applied after each LSTM layer to control the over-fitting of the data and improve the feature extraction ability of the network. The results of 1 st hidden layer (with 100 feature-length) to the input of the 2nd hidden layer (with 50 feature-length), rely on the calculation between the matrices, namely the weight (W) and the bias (b_0). The principle of feature dimension conversion in the LSTM model is described below. Since LSTM is a special RNN, the RNN structure can describe the transfer process of features between two hidden layers. RNN's recursive function:

$$h_t = \tanh\left(W \cdot \left[h_{t-1}, x_t\right] + b_0\right) \quad (8)$$

Where x_t is the input at the current time t, h_{t-1} is the output at the last time, W and b_0 is the weight and bias of the model. Respectively tanh is the activation function, and h_t is the output at the current time t. The shape of the parameters in the RNN is $\boldsymbol{x_t}$: [batch_size, input_size], $\boldsymbol{H_{t-1}}$: [batch_size, Hidden_size], $\boldsymbol{W}$: [input_size+ Hidden_size, Hidden_size], $\boldsymbol{B_0}$: [Hidden_size].

According to $[h_{t-1}, x_t]=[\boldsymbol{x_t}, \boldsymbol{H_{t-1}}]$, The shape obtained by is: [batch_size, input_size+ Hidden_size]; According to $[x_t, h_{t-1}]\cdot W$, the shape calculated by matrix multiplication is [batch_size, Hidden_size]; According to $[x_t, h_{t-1}]\cdot W + b_0$, the shape of $\boldsymbol{H_t}$ calculated by adding $\boldsymbol{B_0}$ is [batch_size, Hidden_size]. With the output of the first hidden layer as the input of the second hidden layer, the parameter values of this model are batch_size = 200, input_size = 100, Hidden_size = 50, and substitute each parameter into the feature dimension that the above analysis process can obtain is 50.

Finally, the output layer, which is used to output the predicted RUL array, is a fully connected layer containing a feed-forward neural network. Since it is a regression problem, the output layer includes a single unit and a dense output layer with a "linear" activation function.

2.2. RUL prediction model framework based on multi-scale sequence and LSTM

When using the LSTM network for RUL prediction, there are generally the following two solutions :

- **Sequence-to-one model**

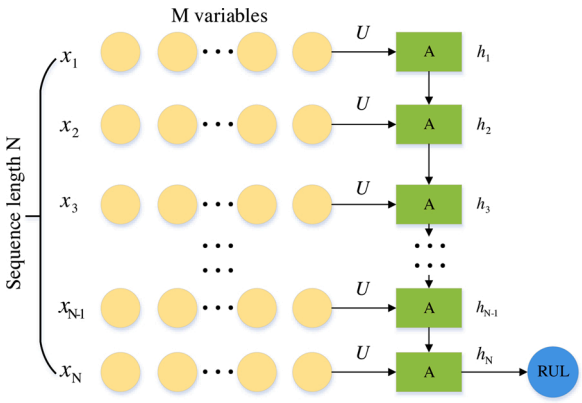


Fig. 3. Input and output process of LSTM.

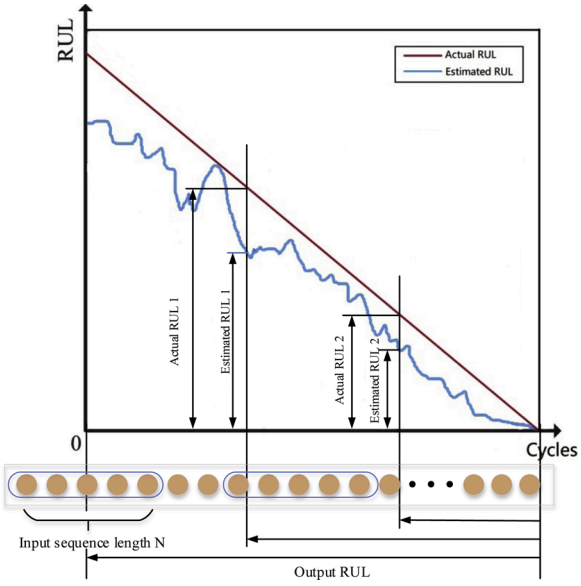


Fig. 4. RUL prediction flowchart.

The data sequence of the entire life cycle is divided into several short sequences of fixed length, which are selected as input with no output in the middle, only the RUL at the last cycle number.

- **Sequence-to-sequence model**

The sequence-to-sequence model is a neural network method using end-to-end sequence learning. This model selects the data sequence of the entire life cycle as input and the RUL sequence composed of RUL tags as output each time, which is generally used for text data analysis.

Considering the change of the sequence length of the whole life cycle, the Sequence-to-sequence model will be more challenging to train. In the meantime, the available time sequence data is usually not the entire life cycle. The model treats each life cycle data as only one input sample, which will cause LSTM to overfit due to sparse samples. Mainly, it is necessary to verify the applicability of the model further when using a model trained with complete life cycle data to perform RUL estimation on non-full life cycle data.

Compared with the sequence-to-sequence model, the applicability of the sequence-to-one model of the data sequences becomes higher. Each complete life cycle data is divided into hundreds of samples to avoid overfitting problems due to sparse samples. Moreover, it is easier to build and train models with input samples of fixed sequence length. Therefore, the sequence-to-one model is selected for RUL prediction in this paper. The input and output process of the sequence-to-one model is shown in Fig. 3. In the sequence-to-one model, N represents the sequence length which is the number of input training samples. Since sequences of different lengths contain different prediction information, different prediction results can be obtained by adjusting the value of N. In this paper, several training indicators are set to measure the model's performance, and the value of the input parameter N can be obtained when the model parameters are optimal.

Next, the RUL prediction process is shown in Fig. 4. After the data preprocessing is completed, the data of fixed sequence length is input into the model to output the estimated RUL value. The information contained in different sequence lengths will also be different, which will affect the model's prediction accuracy. Therefore, this paper studies the impact of multi-scale sequences on the training and prediction performance of the LSTM model.

In this paper, historical monitoring data and real remaining life serve as input, and time sequence prediction of remaining life and related evaluation indicators are taken as output. In this step, the sliding window method is adopted to process time sequence data to prepare 3D format training data with a specific sequence length to fit the neural network on the training set. Then, during the training process, the partial derivative of the error function is solved, and the model parameters are updated by the stochastic gradient descent method to realize the layer-by-layer backpropagation of the error, fine-tuning the model parameters. The performance is evaluated by a cross-validation set. When predicting the remaining service life on the test set, only engine cycles that meet the sequence length are considered, not filling, which means that to perform the test, those engine cycles that are less than the sequence length need to be deleted.

The callbacks parameter in the fit function is used in the model training process to speed up the training process and save the model. Generally, the larger the epochs is, the smaller the final training loss value is, whereas too many iterations lead to overfitting. We expect to stop training when the loss value or accuracy reaches a specific value. Nonetheless, we can't wait or control artificially. The callback mechanism in keras.callbacks handles this problem well for us. Min_delta is a parameter in the callback function, which represents the threshold to trigger the suspension. If the changed value is less than min_delta, the model performance will not be considered. If there is an improvement, stop training the model to prevent overfitting. The callback mechanism in keras can realize the automatic call of the specified function after each iteration of around, which can facilitate us to control the timing of training termination, thereby improving training efficiency and model fitting accuracy.

The detailed process of the MS-LSTM framework proposed in this paper is shown in Fig. 5.

2.3. Evaluation indicators

To quantitatively evaluate the prediction performance of the proposed model, four evaluation indicators are used: Mean Absolute Error (MAE) is used to measure the average absolute error between the predicted value and the actual value. The smaller the MAE, the better the model; In general, the expected value range of R2 is [0,1]. The closer to 1, the better the model fits the data. The advantage of R2 is that the results are normalized, making it easier to see the gap between the models; The Root Mean Square Error (RMSE) can be used to evaluate the accuracy of the model RUL evaluation; Score It can judge the prediction performance of the model RUL. Whether it is an early prediction or a late prediction, there is a corresponding score expression. In particular, RMSE and Score are widely used in related research papers and confirmed by affiliated scholars to evaluate model training performance effectively.

- **MAE** is defined as

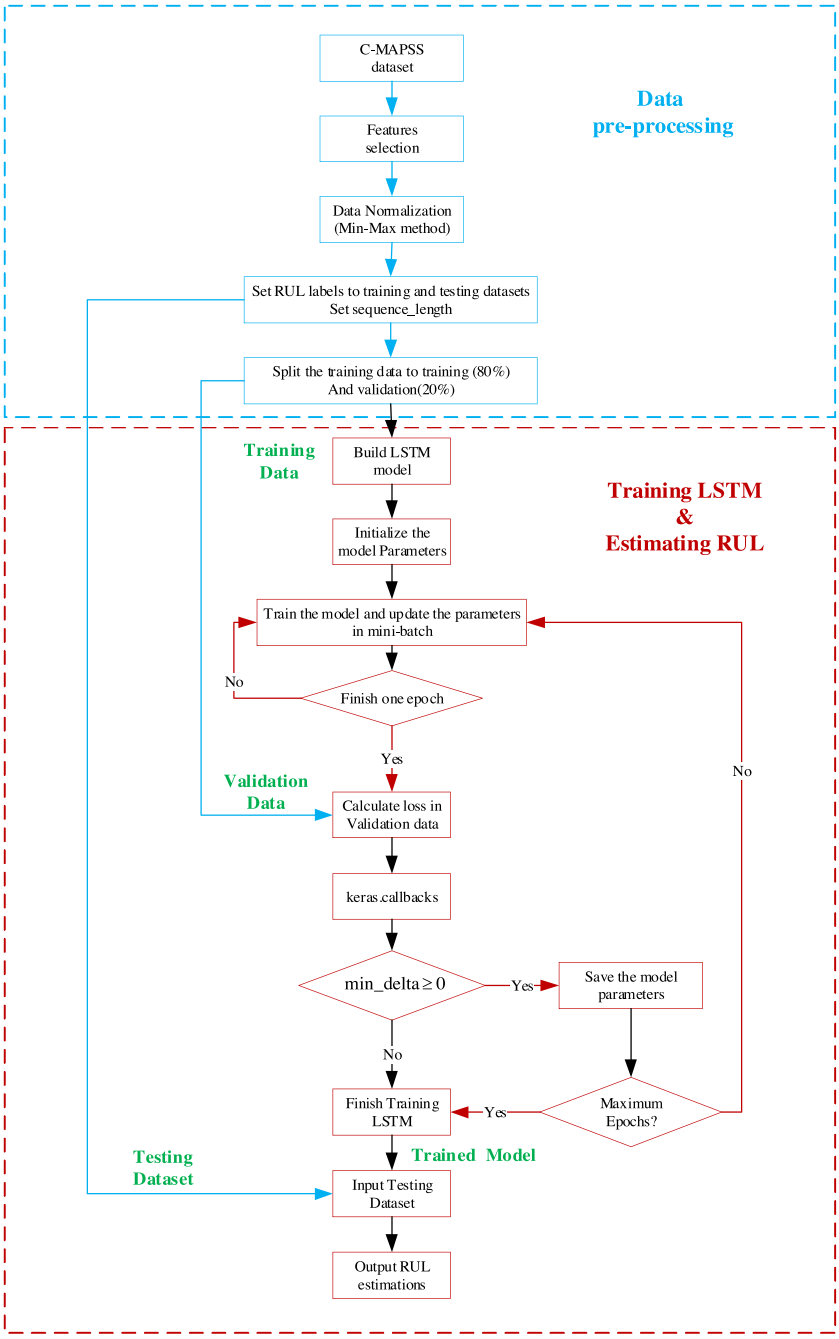


Fig. 5. Aeroengine remaining life prediction based MS-LSTM.

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \widehat{y}_i| \tag{9}$$

- **R2** is given by

$$\mathrm{R2} = 1 - \frac{\sum_{i=1}^{N} w_i(y_i - \widehat{y}_i)^2}{\sum_{i=1}^{N} w_i(y_i - \bar{y})^2} \tag{10}$$

- **RMSE** is another common method to evaluate the accuracy of prediction.

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \widehat{y}_i)^2} \tag{11}$$

- **Score**: The scoring function [23] is defined as

$$Score = \begin{cases} \sum_{i=1}^{N}(e^{-\frac{\widehat{y}_i - y_i}{13}} - 1), when\ \widehat{y}_i < y_i \\ \sum_{i=1}^{N}(e^{\frac{\widehat{y}_i - y_i}{10}} - 1), when\ \widehat{y}_i \geq y_i \end{cases} \tag{12}$$

In the above formula, N is the total number of test data samples; y_i and $\widehat{y}_i$ represent the predicted value and the true value, respectively; w is

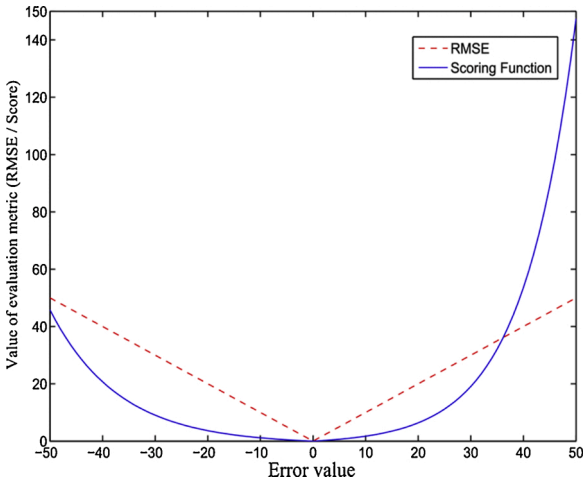


Fig. 6. RMSE versus scoring function (Fig. 6 in [23]).

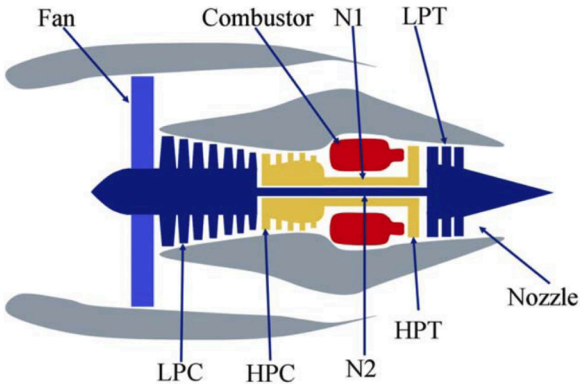


Fig. 7. Diagram of engine simulated in C-MAPSS (Fig. 7 in [24]).

Table 1
Format of C-MAPSS dataset.

Column Number	Contents
1	Engine Unit No.
2	Time in cycles
3	Operational Setting 1
4	Operational Setting 2
5	Operational Setting 3
6	Sensor Measurement 1
⋮	⋮
26	Sensor Measurement 21

the weight; S is the computed *score*.

RMSE and Scoring Function are used to evaluate the performance of RUL prediction. When using the Machine Learning model to predict RUL, late prediction means that the predicted RUL is greater than the actual RUL [23]. Compared with early prediction, late prediction usually leads to more severe consequences. The comparison between the RMSE and the scoring function is shown in Fig. 6. The score has a more significant penalty for late prediction than RMSE, which aligns with the engineering practice. These two indicators are very useful for evaluating model's performance, and test results will be reported according to these two criteria.

Table 2
Description of C-MAPSS dataset.

Column Contents	FD001	FD002	FD003	FD004
Engines in the training set	100	260	100	248
Engines in the test set	100	259	100	249
Training samples	20,631	53,759	24,720	61,249
Test samples	100	259	100	249
Condition number	1	6	1	6
Fault mode number	1	1	2	2

3. Case study

3.1. Data description

In this paper, the proposed method is implemented and evaluated based on the turbofan engine degradation datasets provided in references [24] and [25]. The NASA C-MAPSS dataset is a widely used benchmark data. The degradation data comes from the simulation of the turbofan engine (shown in Fig. 7) in the C-MAPSS platform. It is generated by the C-MAPSS tool, which can simulate various degradations of the same type of engine fleet. The training set mainly includes the sensor parameters of each engine from early health state to failure. In the test set, the degradation process ends sometime before the system failure.

The C-MAPSS dataset includes four subsets: FD001, FD002, FD003, and FD004. They have different operating conditions and failure modes, and each sub-dataset is further divided into a training set and a test set. Table 1 lists the components of the C-MAPSS dataset. Both the training set and the test set consist of 26 columns, which describe the characteristics of the engine unit. The first and second rows represent the engine unit number and degradation cycle of each engine. The third row describes the engine's operating settings. The last 21 columns consist of multivariate time data from 21 sensors. Table 2 is a description of the C-MAPSS dataset. In recent years, this data integration has become a well-known benchmark in the field of RUL prediction. The dataset FD001 will be considered in future experiments.

3.2. Data preprocessing

3.2.1. Sensor selection

The C-MAPSS dataset is composed of multivariate time sequence data from the 21 sensors. Some of 21 sensors have almost constant output during the entire life cycle of the engine, which causes the futility of information to estimate RUL [26]. Therefore, 14 sensor features are selected from the C-MAPSS dataset, corresponding to the 14 sensor values of numbers: S2, S3, S4, S7, S8, S9, S11, S12, S13, S14, S15, S17, S20, and S21.

3.2.2. Normalization

In this paper, the Min-Max data standardization method is adopted to normalize data of training set and test set, and the specific process is shown as follows [24]:

$$\overline{x}_i = \frac{x_i - \min x_i}{\max x_i - \min x_i} \tag{13}$$

Where x_i is the time sequence measured by the i-th sensor, $\overline{x}_i$ is normalized sensor data while $\max x_i$ and $\min x_i$ are the maximum and minimum values of the i-th sensor data before normalization, respectively.

This normalization will ensure the equal contribution of all functions under all working conditions. The normalized data will be within [0,1]. For the sub-dataset of FD001 in C-MAPSS, Min-max normalization has been utilized to achieve the unbiased contribution of each sensor output. Fig. 8 shows data from 14 selected sensors from FD001 before and after

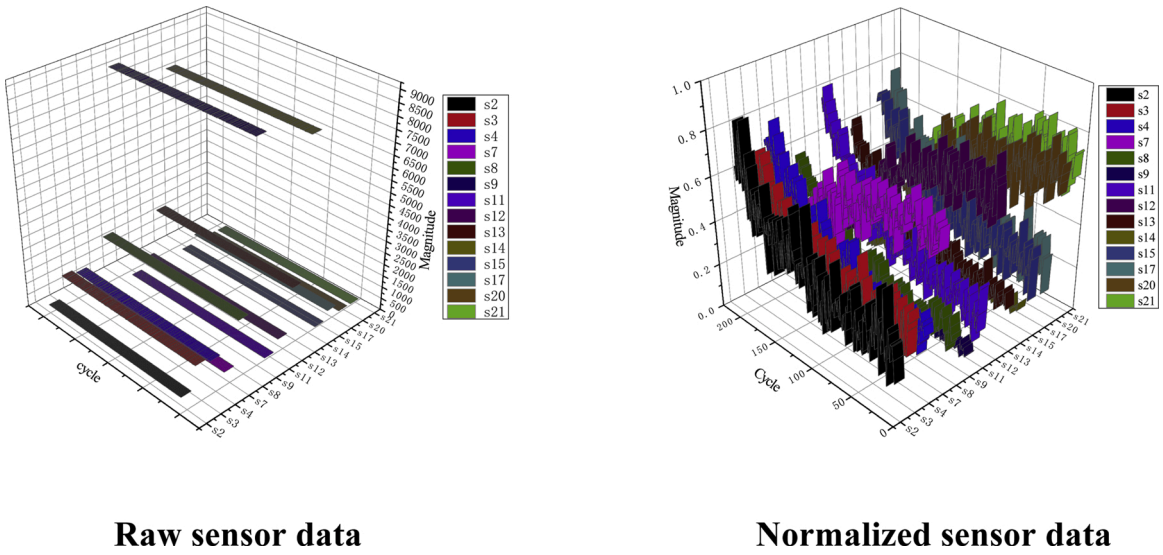


Fig. 8. Illustration of the data from the 14 selected sensors during the whole life cycle of a single engine in the training set of FD001 before and after normalization.

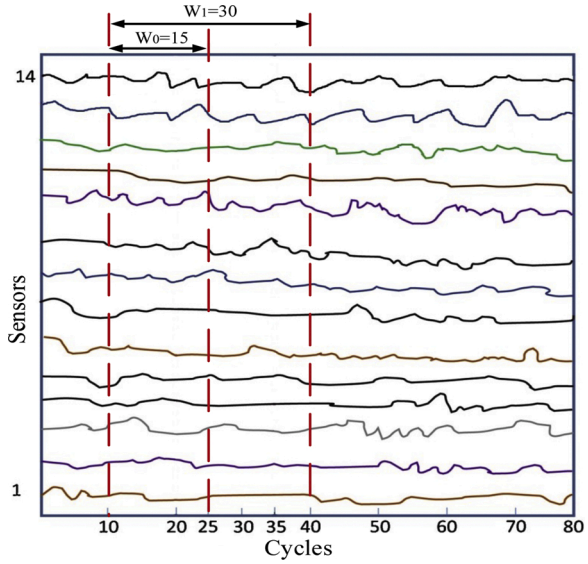


Fig. 9. One training sample with 14 selected features using a time window of length 15 and 30.

normalization.

3.2.3. Data labeling

To train and test the model, it is necessary to mark the training set and test set data. Sliding time windows is applied to define data labels in this paper. In the problem of RUL estimation based on multivariate time sequence, more useful time information can be obtained by applying time window processing to data points sampled on several consecutive timestamps to obtain feature vectors, which may greatly improve prediction performance [27,28]. The sliding time window method can utilize the previous time step to predict the next time step. The time step refers to the length of each feature's information that can be passed to the next feature [29]. Specifically, at any given time step, multivariate sensor data of the current time step and its previous time step within the time window are collected to form a high-dimensional feature vector, which is then input into the prediction model.

The improvement of model performance is attributed to the utilization of helpful time-series information in the time window. On the other hand, the performance degradation is due to the sample introducing irrelevant information when the time window is too small. Moreover, a large window size will result in high-dimensional feature vectors, which increases the computational cost. Given these facts, the time window size used in our research is set to $W_0 = 15$ and $W_1 = 30$ [28]. In addition, the step size of the sliding window is selected as 1. Fig. 9 shows a standardized training data sample from 14 selected sensor features for a single-engine in the dataset FD001 within a time window of sizes 15 and 30.

Table 3
Configuration parameters of the LSTM network.

Epochs	Batch_size	Validation_split	Dropout	LSTM_1 units	LSTM_2 units
100	200	0.2	0.2	100	50

Table 4
Multi-scale sequence division.

Experiment	1	2	3	4	5	6	7
Seq-length	50	55	60	65	70	75	80

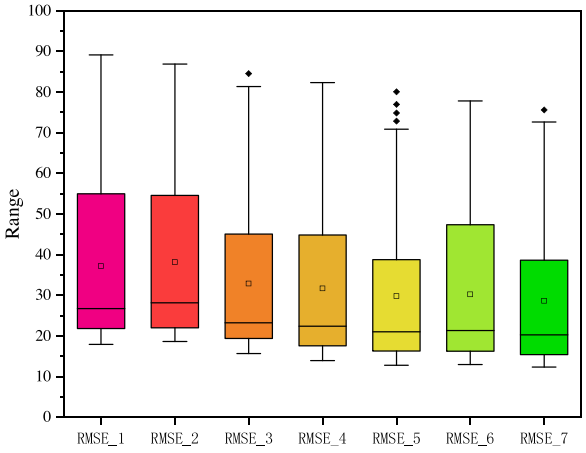


Fig. 10. RMSE box diagram of model training.

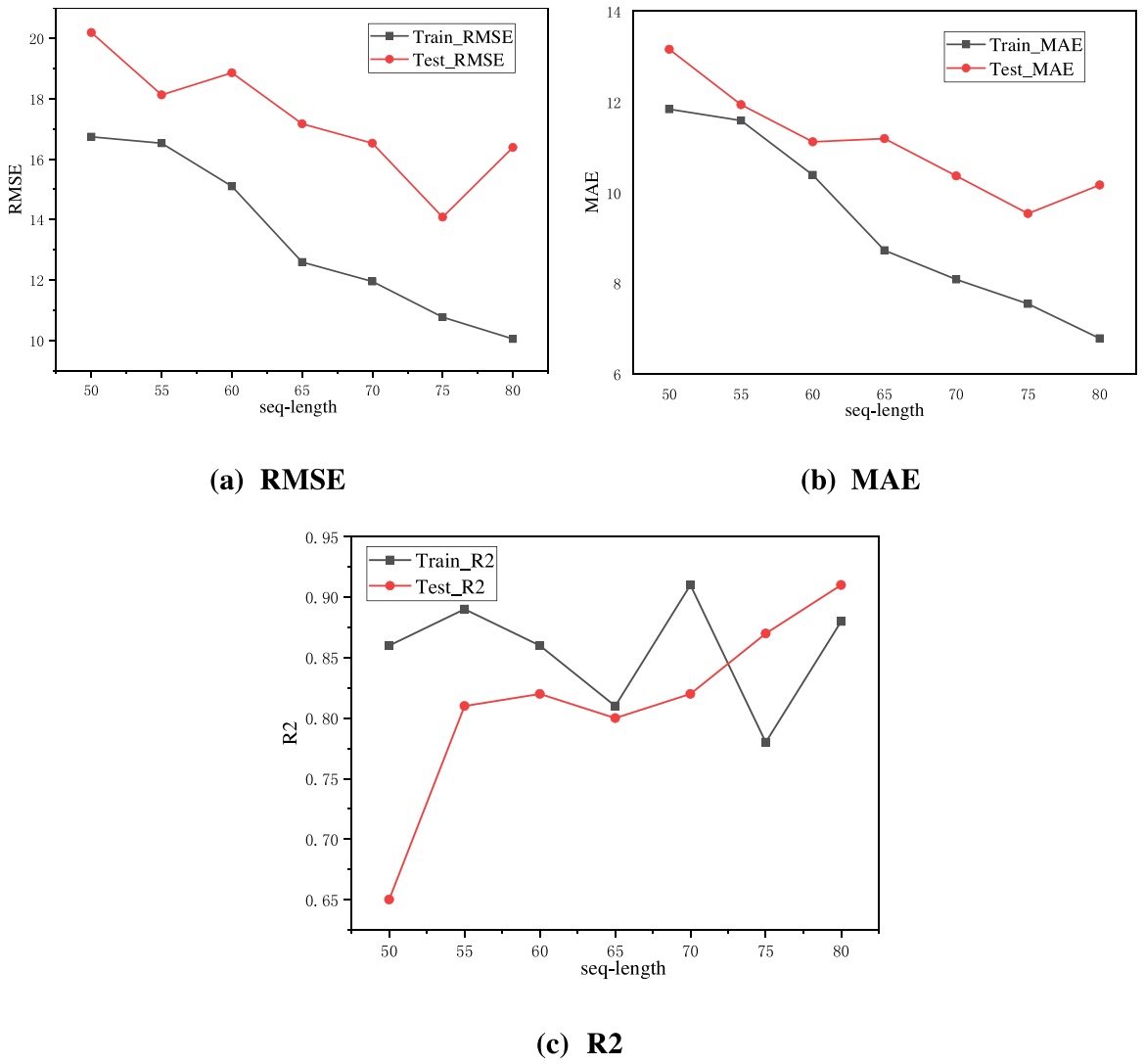


Fig. 11. Comparison of RMSE, MAE and R2 results in training set and test set.

3.3. RUL prediction

3.3.1. Experimental setting and description

When constructing an LSTM network, it is necessary to define the hidden layer, the number of neurons, loss function, maximum training times (Epochs), Batch_size, initial values of model parameters of LSTM, etc. The relevant parameters are shown in Table 3. In addition, the Mean Squares Error (MSE) function is defined as the loss, and the *rmsprop* optimizer with an initial learning rate of 0.01 is employed. The proposed LSTM structure has been introduced in Section 2.2. The sequence length N is divided as shown in Table 4. Next, the relationship between multi-scale sequences and RUL prediction performance is discussed.

3.3.2. Experimental results

3.3.2.1. Evaluation and comparison of training performance and test performance. According to the multi-scale sequence division described in Table 4, the RMSE box diagram of the training stage obtained with seven different scale sequences is shown in Fig. 10. In the training stage, RMSE has good results in experiments 5, 6, and 7, which shows that the model has excellent and powerful training performance and good robustness. In addition, Fig. 11 (a) describes the changing trend of RMSE in each group of experiments during the training and the test stages, respectively. It can be clearly seen that the performance of model training will first improve as the sequence scale increases in most cases, then worsen when the sequence scale becomes too large. At the same time, performance is optimal when seq-length = 80 when Train-RMSE = 10.05. In the test phase, although the RMSE augments slightly with the increase of sequence length N at the beginning. However, RMSE reaches the lowest value at $N = 75$, which indicates that the test performance and training performance of the model reaches the optimal value at the same time; The same result can be obtained from Fig. 11 (b), when $N = 75$, the optimal values of the performance indexes of the test sets of MAE are 9.54; It can be seen from Fig. 11(c) that in the test phase, both experiments 6 and 7 have achieved good results, but this article takes the value of RMSE as the main reference. Based on the above analysis process, the model of experiment 6 has the best performance.

Fig. 12 describes the evaluation indexes (loss, MAE, R2, RMSE) of the training set and the verification set in Experiment 6. When the training reaches 100 Epochs, the MS-LSTM model obtains the optimal parameters and stops training. Fig. 12 shows that as time increases, the loss of training and testing gradually decreases, and the loss error between the training and the verification sets is small. When the training reaches 25 epochs, the error decline trend of the model fluctuates obviously, especially the amplitude of MAE fluctuates wildly, which is related to the two hidden layers set in this article. After the model is trained to 75 epochs, the changing trend of various indicators tends to be stable. It is worth noting that R2 has a negative value within the first 25 epochs, i.e.,

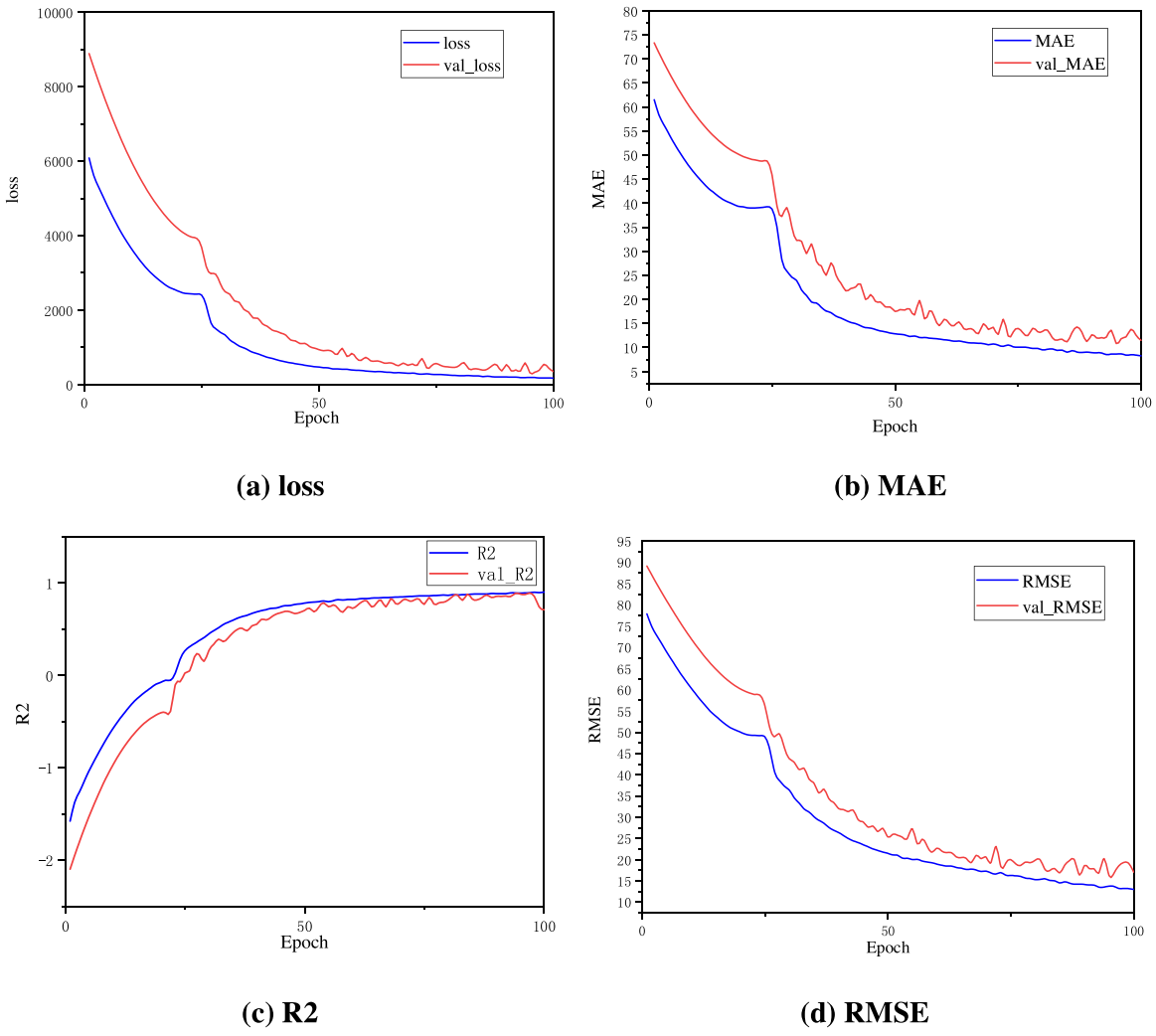


Fig. 12. Performance evaluation indexes (loss, MAE, R2, RMSE) of training set in experiment 6.

Table 5
RUL prediction output related parameters.

Experiment	Seq-length	RMSE	Score	MAE	R2	Epoch
1	50	20.19	16388.82	13.16	0.65	80
2	55	18.13	6133.15	11.94	0.81	75
3	60	18.86	31549.57	11.12	0.82	100
4	65	17.17	1162.86	11.19	0.79	99
5	70	16.53	3227.65	10.37	0.82	100
6	75	14.08	303.79	9.54	0.87	100
7	80	16.39	628.96	10.17	0.91	100

the model's performance is too low at this time, which is because that the prediction error at the initial stage of model fitting is greater than the average error. However, with the increase of training times, the model's performance is significantly improved, and R2 reached more than 0.9. At this time, RMSE reaches a stable state and remains at a low level. It is shown that the proposed MS-LSTM has good results in model parameter fitting by comparing the change trends of the indexed in the training set and verification set.

3.3.2.2. Analysis of RUL prediction results. After model training, the test set is brought into the trained model to predict the remaining service lifetime sequence. The relevant parameters of the RUL prediction output are shown in Table 5, where the gray part shows the best experimental result. When predicting RUL, it is necessary to delete those engine ids

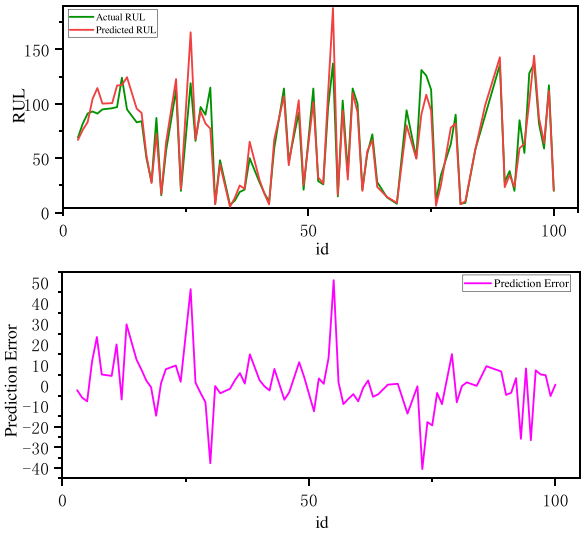


Fig. 13. Comparison of predicted RUL values vs real RUL values for dataset FD001.

Table 6
Comparison between different methods on the FD001 test set.

Methods	RMSE	Score
Vanilla LSTM [30]	19.74	N/A
RF [31]	17.91	479.75
GAN [32]	16.91	N/A
Deep LSTM [33]	16.14	1862
Bi-LSTM [34]	15.42	N/A
DBN [35]	15.21	417.59
Standard RNN [36]	15.07	367.31
MONBNE [37]	15.04	334.23
ABDL [23]	14.53	322.44
Proposed MS-LSTM	**14.08**	**303.79**

whose sequence number is less than the sequence length N. For experiment 6, the RUL of these 80 engines that meets the requirements is predicted and evaluated. Fig. 13 shows the RUL prediction results of the subset FD001. It can be observed that, except for the 4 Engines, the predicted RUL value is very close to the real RUL value. The RUL values predicted by Engine 55 and Engine 26 are substantially larger than the real RUL values. The RUL values predicted by Engine 30 and Engine 73 are significantly smaller than the real RUL values. In all prediction results, the maximum absolute error is 51.05, the minimum is 0.17, and the average error is 0.44.

Most errors are related to prediction greater than the actual RUL value, and the error fluctuation is within a reasonable range, indicating that the proposed MS-LSTM model has good prediction accuracy and robustness.

3.3.3. Comparison with other methods

In recent years, C-MAPSS data has been widely concerned and has received many achievements. To further evaluate the proposed method, some latest research methods are selected for comparison. In this paper, the machine learning algorithm within five years is used as a comparison. The proposed method has been compared with several existing methods in Table 6. Specifically, Deep LSTM achieved a higher score than the RF, which certain outliers could cause in predicting the RUL. The standard RNN can generally work well on the FD001 test set, but it needs to contain more layers. It is easier to utilize RNNs in the dedicated libraries of TensorFlow/Keras, which has the disadvantage of computationally expensive network structures. It is worth noting that the prediction accuracy of Bi-LSTM is slightly higher than that of Deep LSTM. Compared with the latest results, it shows that the proposed MS-LSTM has a superior performance in solving the RUL prediction problem of aircraft engines.

4. Conclusion

In this paper, a novel RUL prediction framework combining multi-scale sequences and LSTM is proposed. First, the framework, using sliding time windows to generate training and test labels, constructs a deep LSTM network that can map the HI curve of sensor data to RUL values directly with two hidden layers. Secondly, the multi-scale sequence model prediction effect is evaluated, and the optimal RUL prediction result is obtained. Meanwhile, the best model parameters are output by the keras.callbacks mechanism, which saves the training time and improves the prediction accuracy. The best relevant data parameters are obtained by comparing the evaluation indicators output by different experiments, where RMSE = 14.08, Score = 303.79, MAE = 9.54, and R2 = 0.87, respectively. The model results of the framework presented in this paper have proved to be accurate and computationally efficient, mainly when applied to large datasets. Furthermore, the proposed method shows the best performance compared with other latest methods, especially applicable for Cyber-Physical Systems. The framework has the advantages of precise, compact, and strong applicability.

The proposed framework, having achieved good prognostic results, is versatile and extensible. However, the model framework still needs further optimization. Future work will consider choosing a better neural structure, and taking the time window and sequence length as two variables, and analyzing its relationship with the prediction accuracy.

Data availability

No data was used for the research described in the article.
Data will be made available on request.

Declaration of Competing Interest

The authors report no declarations of interest.

CRediT authorship contribution statement

Ruiguan Lin: Methodology, Data curation, Writing - original draft, Methodology, Writing - review & editing, Software, Visualization, Formal analysis. **Yaowei Yu:** Writing - review & editing. **Huawei Wang:** Conceptualization, Funding acquisition, Supervision, Project administration. **Changchang Che:** Data curation, Investigation, Validation. **Xiaomei Ni:** Resources.

Acknowledgment

This research is supported by National Natural Science Foundation of China (No. U1833110).

References

[1] K.T.P. Nguyen, K. Medjaher, A new dynamic predictive maintenance framework using deep learning for failure prognostics, Reliab. Eng. Syst. Saf. 188 (2019) 251–262, https://doi.org/10.1016/j.ress.2019.03.018.
[2] A. Diez-Olivan, J. Del Ser, D. Galar, B. Sierra, Data fusion and machine learning for industrial prognosis: trends and perspectives towards Industry 4.0, Inf. Fusion 50 (2019) 92–111, https://doi.org/10.1016/j.inffus.2018.10.005.
[3] J. Lee, B. Bagheri, H.-A. Kao, A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems, Manuf. Lett. 3 (2015) 18–23, https://doi.org/10.1016/j.mfglet.2014.12.001.
[4] J. Lee, H. Davari, J. Singh, V. Pandhare, Industrial Artificial Intelligence for industry 4.0-based manufacturing systems, Manuf. Lett. 18 (2018) 20–23, https://doi.org/10.1016/j.mfglet.2018.09.002.
[5] Z. Huang, Z. Xu, X. Ke, W. Wang, Y. Sun, Remaining useful life prediction for an adaptive skew-Wiener process model, Mech. Syst. Signal Process. 87 (2017) 294–306, https://doi.org/10.1016/j.ymssp.2016.10.027.
[6] A. Elsheikh, S. Yacout, M.-S. Ouali, Bidirectional handshaking LSTM for remaining useful life prediction, Neurocomputing 323 (2019) 148–156, https://doi.org/10.1016/j.neucom.2018.09.076.
[7] Z. Chen, S. Deng, X. Chen, C. Li, R.-V. Sanchez, H. Qin, Deep neural networks-based rolling bearing fault diagnosis, Microelectron. Reliab. 75 (2017) 327–333, https://doi.org/10.1016/j.microrel.2017.03.006.
[8] L. Guo, N. Li, F. Jia, Y. Lei, J. Lin, A recurrent neural network based health indicator for remaining useful life prediction of bearings, Neurocomputing 240 (2017) 98–109, https://doi.org/10.1016/j.neucom.2017.02.045.
[9] H. Liu, J. Zhou, Y. Zheng, W. Jiang, Y. Zhang, Fault diagnosis of rolling bearings with recurrent neural network-based autoencoders, ISA Trans. 77 (2018) 167–178, https://doi.org/10.1016/j.isatra.2018.04.005.
[10] J. Schmidhuber, F. Cummins, Learning to Forget: Continual Prediction with LSTM, (n.d.) 6.
[11] Y. Bengio, P. Simard, P. Frasconi, Learning long-term dependencies with gradient descent is difficult, IEEE Trans. Neural Netw. 5 (1994) 157–166, https://doi.org/10.1109/72.279181.
[12] P.R. de O. da Costa, A. Akçay, Y. Zhang, U. Kaymak, Remaining useful lifetime prediction via deep domain adaptation, Reliab. Eng. Syst. Saf. 195 (2020) 106682, https://doi.org/10.1016/j.ress.2019.106682.
[13] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[14] A. Graves, J. Schmidhuber, Framewise phoneme classification with bidirectional LSTM and other neural network architectures, Neural Netw. 18 (2005) 602–610, https://doi.org/10.1016/j.neunet.2005.06.042.
[15] A.Z. Hinchi, M. Tkiouat, Rolling element bearing remaining useful life estimation based on a convolutional long-short-term memory network, Procedia Comput. Sci. 127 (2018) 123–132, https://doi.org/10.1016/j.procs.2018.01.106.
[16] M. Yuan, Y. Wu, L. Lin, Fault diagnosis and remaining useful life estimation of aero engine using LSTM neural network, in: 2016 IEEE International Conference on Aircraft Utility Systems (AUS), IEEE, Beijing, China, 2016, pp. 135–140, https://doi.org/10.1109/AUS.2016.7748035.

[17] D. Laredo, Z. Chen, O. Schütze, J.-Q. Sun, A neural network-evolutionary computational framework for remaining useful life estimation of mechanical systems, Neural Netw. 116 (2019) 178–187, https://doi.org/10.1016/j.neunet.2019.04.016.
[18] A. Al-Dulaimi, S. Zabihi, A. Asif, A. Mohammadi, A multimodal and hybrid deep neural network model for remaining Useful Life estimation, Comput. Ind. 108 (2019) 186–196, https://doi.org/10.1016/j.compind.2019.02.004.
[19] Y. Cao, Y. Ding, M. Jia, R. Tian, A novel temporal convolutional network with residual self-attention mechanism for remaining useful life prediction of rolling bearings, Reliab. Eng. Syst. Saf. 215 (2021) 107813, https://doi.org/10.1016/j.ress.2021.107813.
[20] T. Han, J. Pang, A.C.C. Tan, Remaining useful life prediction of bearing based on stacked autoencoder and recurrent neural network, J. Manuf. Syst. 61 (2021) 576–591, https://doi.org/10.1016/j.jmsy.2021.10.011.
[21] J. Medina-Quero, S. Zhang, C. Nugent, M. Espinilla, Ensemble classifier of long short-term memory with fuzzy temporal windows on binary sensors for activity recognition, Expert Syst. Appl. 114 (2018) 441–453, https://doi.org/10.1016/j.eswa.2018.07.068.
[22] X. Li, L. Peng, X. Yao, S. Cui, Y. Hu, C. You, T. Chi, Long short-term memory neural network for air pollutant concentration predictions: method development and evaluation, Environ. Pollut. 231 (2017) 997–1004, https://doi.org/10.1016/j.envpol.2017.08.114.
[23] Z. Chen, M. Wu, R. Zhao, F. Guretno, R. Yan, X. Li, Machine remaining useful life prediction via an attention-based deep learning approach, IEEE Trans. Ind. Electron. 68 (2021) 2521–2531, https://doi.org/10.1109/TIE.2020.2972443.
[24] A. Saxena, K. Goebel, D. Simon, N. Eklund, Damage propagation modeling for aircraft engine run-to-failure simulation, in: 2008 International Conference on Prognostics and Health Management, IEEE, Denver, CO, USA, 2008, pp. 1–9, https://doi.org/10.1109/PHM.2008.4711414.
[25] A. Saxena, K. Goebel, Turbofan Engine Degradation Simulation Data Set, NASA Ames Prognostics Data Repository [Online]. Available:, 2008 http://ti.arc.nasa.gov/project/prognostic-data-repository.
[26] A. Al-Dulaimi, S. Zabihi, A. Asif, A. Mohammadi, A multimodal and hybrid deep neural network model for remaining Useful Life estimation, Comput. Ind. 108 (2019) 186–196, https://doi.org/10.1016/j.compind.2019.02.004.
[27] T. Xia, Y. Song, Y. Zheng, E. Pan, L. Xi, An ensemble framework based on convolutional bi-directional LSTM with multiple time windows for remaining useful life estimation, Comput. Ind. 115 (2020) 103182, https://doi.org/10.1016/j.compind.2019.103182.
[28] C. Zhang, P. Lim, A.K. Qin, K.C. Tan, Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics, IEEE Trans. Neural Netw. Learn. Syst. 28 (2017) 2306–2318, https://doi.org/10.1109/TNNLS.2016.2582798.
[29] B. Zhang, S. Zhang, W. Li, Bearing performance degradation assessment using long short-term memory recurrent network, Comput. Ind. 106 (2019) 14–29, https://doi.org/10.1016/j.compind.2018.12.016.
[30] Y. Wu, M. Yuan, S. Dong, L. Lin, Y. Liu, Remaining useful life estimation of engineered systems using vanilla LSTM neural networks, Neurocomputing 275 (2018) 167–179, https://doi.org/10.1016/j.neucom.2017.05.063.
[31] J. Li, X. Li, D. He, A directed acyclic graph network combined with CNN and LSTM for remaining useful life prediction, IEEE Access 7 (2019) 75464–75475, https://doi.org/10.1109/ACCESS.2019.2919566.
[32] D. Verstraete, E. Droguett, M. Modarres, A deep adversarial approach based on multi-sensor fusion for semi-supervised remaining useful life prognostics, Sensors 20 (2019) 176, https://doi.org/10.3390/s20010176.
[33] S. Zheng, K. Ristovski, A. Farahat, C. Gupta, Long short-term memory network for remaining useful life estimation, in: 2017 IEEE International Conference on Prognostics and Health Management (ICPHM), IEEE, Dallas, TX, USA, 2017, pp. 88–95, https://doi.org/10.1109/ICPHM.2017.7998311.
[34] J. Zhang, P. Wang, R. Yan, R.X. Gao, Long short-term memory for machine remaining life prediction, J. Manuf. Syst. 48 (2018) 78–86, https://doi.org/10.1016/j.jmsy.2018.05.011.
[35] H. Liu, Z. Liu, W. Jia, X. Lin, Remaining useful life prediction using a novel feature-attention-based end-to-end approach, IEEE Trans. Ind. Inf. 17 (2021) 1197–1207, https://doi.org/10.1109/TII.2020.2983760.
[36] S. Zhao, Y. Zhang, S. Wang, B. Zhou, C. Cheng, A recurrent neural network approach for remaining useful life prediction utilizing a novel trend features construction method, Measurement 146 (2019) 279–288, https://doi.org/10.1016/j.measurement.2019.06.004.
[37] H. Wang, M. Peng, Z. Miao, Y. Liu, A. Ayodeji, C. Hao, Remaining useful life prediction techniques for electric valves based on convolution auto encoder and long short term memory, ISA Trans. 108 (2021) 333–342, https://doi.org/10.1016/j.isatra.2020.08.031.



Ruiguan Lin received the M.S. degree in aviation engineering from Shenyang Aerospace University, Shenyang, China, in 2019. He is currently pursuing the Ph.D. degree at the College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His current research interests include prognostic and health management, fault diagnosis and aircraft reliability engineering.



Yaowei Yu received the M.S. degree in Mechanical Engineering at College of Field Engineering from Army Engineering University, Nanjing, China, in 2018. He is currently pursuing the Ph.D. degree at the college of civil aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His current research interests include evolutionary computation, multi-objective optimization, multi-UAV collaborative control technology.



Huawei Wang is a professor of the School of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China. She received his Ph.D. from National University of Defense Technology, Changsha, China. Her research interests include prognostic and health management, fault diagnosis and aircraft reliability engineering.



Changchang Che received the M.S. degree in vehicle operation engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2018. He is currently pursuing the Ph.D. degree at the College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His current research interests include fault diagnosis, vibration signal processing and intelligent maintenance technology.



Xiaomei Ni received the M.S. degree in vehicle operation engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2018. She is currently pursuing the Ph.D. degree at the College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing, China. Her current research interests include civil aviation risk control and management.